

第51章 设置 FLASH 的读写保护及解除

本章参考资料：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库说明文档《stm32f4xx_dsp_stdperiph_lib_um.chm》以及《Proprietary code read-out protection on microcontrollers》。

51.1 选项字节与读写保护

在实际发布的产品中，在 STM32 芯片的内部 FLASH 存储了控制程序，如果不作任何保护措施的话，可以使用下载器直接把内部 FLASH 的内容读取回来，得到 bin 或 hex 文件格式的代码拷贝，别有用心的厂商即可利用该代码文件山寨产品。为此，STM32 芯片提供了多种方式保护内部 FLASH 的程序不被非法读取，但在默认情况下该保护功能是不开启的，若要开启该功能，需要改写内部 FLASH 选项字节(Option Bytes)中的配置。

51.1.1 选项字节的内容

选项字节是一段特殊的 FLASH 空间，STM32 芯片会根据它的内容进行读写保护、复位电压等配置，选项字节的构成见表 51-1。

表 51-1 选项字节的构成

地址	[63:16]	[15:0]
0x1FFF C000	保留	ROP 和用户选项字节 (RDP & USER)
0x1FFF C008	保留	扇区 0 到 11 的写保护 nWRP 位
0x1FFE C000	保留	保留
0x1FFE C008	保留	扇区 12 到 23 的写保护 nWRP 位

选项字节具体的数据位配置说明见表 51-2。

表 51-2 选项字节具体的数据位配置说明

选项字节（字，地址 0x1FFF C000）	
RDP：读保护选项字节。 读保护用于保护 Flash 中存储的软件代码。	
位 15:8	0xAA：级别 0，无保护 其它值：级别 1，存储器读保护（调试功能受限） 0xCC：级别 2，芯片保护（禁止调试和从 RAM 启动）
USER：用户选项字节 此字节用于配置以下功能： 选择看门狗事件：硬件或软件 进入停止模式时产生复位事件 进入待机模式时产生复位事件	
位 7	nRST_STDBY 0：进入待机模式时产生复位 1：不产生复位
位 6	nRST_STOP 0：进入停止模式时产生复位 1：不产生复位

位 5	WDG_SW 0: 硬件看门狗 1: 软件看门狗
位 4	0x1: 未使用
位 3:2	BOR_LEV: BOR 复位级别 这些位包含释放复位信号所需达到的供电电压阈值。 通过对这些位执行写操作, 可将新的 BOR 级别值编程到 Flash。 00: BOR 级别 3 (VBOR3), 复位阈值电压为 2.70 V 到 3.60 V 01: BOR 级别 2 (VBOR2), 复位阈值电压为 2.40 V 到 2.70 V 10: BOR 级别 1 (VBOR1), 复位阈值电压为 2.10 V 到 2.40 V 11: BOR 关闭 (VBOR0), 复位阈值电压为 1.8 V 到 2.10 V
位 1:0	0x1: 未使用
选项字节 (字, 地址 0x1FFF C008)	
位 15	SPMOD: 选择 nWPRi 位的模式 0: nWPRi 位用于写保护(默认) 1: nWPRi 位用于代码读出保护(Proprietary code readout protection, PCROP)
位 14	DB1M: 设置内部 FLASH 为 1MB 的产品的双 Bank 模式 0: 单个 Bank 的模式 1: 使用两个 Bank 的模式
位 13:12	0xF: 未使用
nWRP: Flash 写保护选项字节。 扇区 0 到 11 可采用写保护。	
位 11:0	nWRPi (i 值为 0-11, 对应 0-11 扇区的保护设置): 0: 开启所选扇区的写保护 1: 关闭所选扇区的写保护
选项字节 (字, 地址 0x1FFE C000)	
位 15:0	0xFF: 未使用
选项字节 (字, 地址 0x1FFE C008)	
位 15:12	0xF: 未使用
nWRP: Flash 写保护选项字节。 扇区 12 到 23 可采用写保护。	
位 11:0	nWRPi: 0: 开启扇区 i 的写保护。 1: 关闭扇区 i 的写保护。

我们主要讲解选项字节配置中的 RDP 位和 PCROP 位, 它们分别用于配置读保护级别及代码读出保护。

51.1.2 RDP 读保护级别

修改选项字节的 RDP 位的值可设置内部 FLASH 为以下保护级别:

- ❑ 0xAA: 级别 0, 无保护

这是 STM32 的默认保护级别, 它没有任何读保护, 读取内部 FLASH 及“备份 SRAM”的内容都没有任何限制。(注意这里说的“备份 SRAM”是指 STM32 备份域的 SRAM 空间, 不是指主 SRAM, 下同)

- ❑ 其它值: 级别 1, 使能读保护

把 RDP 配置成除 0xAA 或 0xCC 外的任意数值，都会使能级别 1 的读保护。在这种保护下，若使用调试功能(使用下载器、仿真器)或者从内部 SRAM 自举时都不能对内部 FLASH 及备份 SRAM 作任何访问(读写、擦除都被禁止)；而如果 STM32 是从内部 FLASH 自举时，它允许对内部 FLASH 及备份 SRAM 的任意访问。

也就是说，在级别 1 模式下，任何尝试从外部访问内部 FLASH 内容的操作都被禁止，例如无法通过下载器读取它的内容，或编写一个从内部 SRAM 启动的程序，若该程序读取内部 FLASH，会被禁止。而如果是芯片自己访问内部 FLASH，是完全没有问题的，例如前面的“读写内部 FLASH”实验中的代码自己擦写内部 FLASH 空间的内容，即使处于级别 1 的读保护，也能正常擦写。

当芯片处于级别 1 的时候，可以把选项字节的 RDP 位重新设置为 0xAA，恢复级别 0。在恢复到级别 0 前，芯片会自动擦除内部 FLASH 及备份 SRAM 的内容，即降级后原内部 FLASH 的代码会丢失。在级别 1 时使用 SRAM 自举的程序也可以访问选项字节进行修改，所以如果原内部 FLASH 的代码没有解除读保护的操作时，可以给它加载一个 SRAM 自举的程序进行保护降级，后面我们将会进行这样的实验。

❑ 0xCC：级别 2，禁止调试

把 RDP 配置成 0xCC 值时，会进入最高级别的读保护，且设置后无法再降级，它会永久禁止用于调试的 JTAG 接口(相当于熔断)。在该级别中，除了具有级别 1 的所有保护功能外，进一步禁止了从 SRAM 或系统存储器的自举(即平时使用的串口 ISP 下载功能也失效)，JTAG 调试相关的功能被禁止，选项字节也不能被修改。它仅支持从内部 FLASH 自举时对内部 FLASH 及 SRAM 的访问(读写、擦除)。

由于设置了级别 2 后无法降级，也无法通过 JTAG、串口 ISP 等方式更新程序，所以使用这个级别的保护时一般会在程序中预留“后门”以更新应用程序，若程序中没有预留后门，芯片就无法再更新应用程序了。所谓的“后门”是一种 IAP 程序(In Application Program)，它通过某个通讯接口获取将要更新的程序内容，然后利用内部 FLASH 擦写操作把这些内容烧录到自己的内部 FLASH 中，实现应用程序的更新。

不同级别下的访问限制见图 51-1。

存储区	保护级别	调试功能， 从 RAM 或系统存储器自举			从 Flash 自举		
		读	写	擦除	读	写	擦除
主 Flash 和备份 SRAM	级别 1	否		否 ⁽¹⁾	是		
	级别 2	否			是		
选项字节	级别 1	是			是		
	级别 2	否			否		
OTP	级别 1	否		NA	是		NA
	级别 2	否		NA	是		NA

1. 只有在 RDP 从级别 1 更改为级别 0 时，才会擦除主 Flash 和备份 SRAM。OTP 区域保持不变。

图 51-1 不同级别下的访问限制

不同保护级别之间的状态转换见图 51-2。

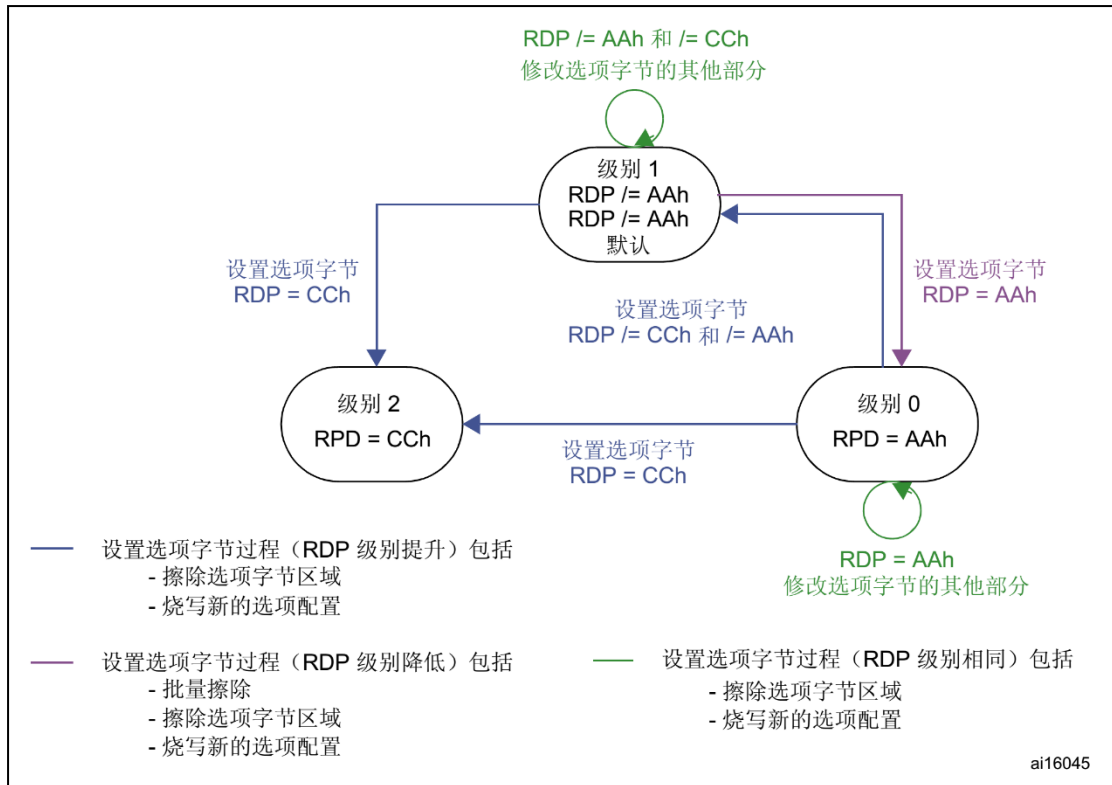


图 51-2 不同级别间的状态转换

51.2 修改选项字节的过程

修改选项字节的内容可修改各种配置，但是，当应用程序运行时，无法直接通过选项字节的地址改写它们的内容，例如，接使用指针操作地址 0x1FFFC0000 的修改是无效的。要改写其内容时必须设置寄存器 FLASH_OPTCR 及 FLASH_OPTCR1 中的对应数据位，寄存器的与选项字节对应位置见图 51-3 及图 51-4，详细说明请查阅《STM32 参考手册》。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SPR MOD	DB1M	Reserved		nWRP[11:0]											
rw	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDP[7:0]								nRST_STDBY	nRST_STOP	WDG_SW	BFB2	BOR_LEV	OPTSTRT	OPTLOCK	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rs	rs

图 51-3 FLASH_OPTCR 寄存器说明(nWRP 表示 0-11 扇区)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				nWRP[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

图 51-4 FLASH_OPTCR1 寄存器说明(nWRP 表示 12-23 扇区)

默认情况下，FLASH_OPTCR 寄存器中的第 0 位 OPTLOCK 值为 1，它表示选项字节被上锁，需要解锁后才能进行修改，当寄存器的值设置完成后，对 FLASH_OPTCR 寄存器

中的第 1 位 OPTSTRT 值设置为 1，硬件就会擦除选项字节扇区的内容，并把 FLASH_OPTCR/1 寄存器中包含的值写入到选项字节。

所以，修改选项字节的配置步骤如下：

- (1) 解锁，在 Flash 选项密钥寄存器 (FLASH_OPTKEYR) 中写入 OPTKEY1 = 0x0819 2A3B；接着在 Flash 选项密钥寄存器 (FLASH_OPTKEYR) 中写入 OPTKEY2 = 0x4C5D 6E7F。
- (2) 检查 FLASH_SR 寄存器中的 BSY 位，以确认当前未执行其它 Flash 操作。
- (3) 在 FLASH_OPTCR 和/或 FLASH_OPTCR1 寄存器中写入选项字节值。
- (4) 将 FLASH_OPTCR 寄存器中的选项启动位 (OPTSTRT) 置 1。
- (5) 等待 BSY 位清零，即写入完成。

51.3 操作选项字节的库函数

为简化编程，STM32 标准库提供了一些库函数，它们封装了修改选项字节时操作寄存器的过程。

1. 选项字节解锁、上锁函数

对选项字节解锁、上锁的函数见代码清单 51-1。

代码清单 51-1 选项字节解锁、上锁

```
1
2 #define FLASH_OPT_KEY1          ((uint32_t)0x08192A3B)
3 #define FLASH_OPT_KEY2          ((uint32_t)0x4C5D6E7F)
4
5 /**
6  * @brief  Unlocks the FLASH Option Control Registers access.
7  * @param  None
8  * @retval None
9  */
10 void FLASH_OB_Unlock(void)
11 {
12     if((FLASH->OPTCR & FLASH_OPTCR_OPTLOCK) != RESET)
13     {
14         /* Authorizes the Option Byte register programming */
15         FLASH->OPTKEYR = FLASH_OPT_KEY1;
16         FLASH->OPTKEYR = FLASH_OPT_KEY2;
17     }
18 }
19
20 /**
21  * @brief  Locks the FLASH Option Control Registers access.
22  * @param  None
23  * @retval None
24  */
25 void FLASH_OB_Lock(void)
26 {
27     /* Set the OPTLOCK Bit to lock the FLASH Option Byte Registers access */
28     FLASH->OPTCR |= FLASH_OPTCR_OPTLOCK;
29 }
```

解锁的时候，它对 FLASH_OPTCR 寄存器写入两个解锁参数，上锁的时候，对 FLASH_OPTCR 寄存器的 FLASH_OPTCR_OPTLOCK 位置 1。

2. 设置读保护级别

解锁后设置选项字节寄存器的 RDP 位可调用 FLASH_OB_RDPConfig 完成，见代码清单 51-2。

代码清单 51-2 设置读保护级别

```
1 /**
2  * @brief Sets the read protection level.
3  * @param OB_RDP: specifies the read protection level.
4  *           This parameter can be one of the following values:
5  *           @arg OB_RDP_Level_0: No protection
6  *           @arg OB_RDP_Level_1: Read protection of the memory
7  *           @arg OB_RDP_Level_2: Full chip protection
8  *
9  * /\ Warning /\ When enabling OB_RDP level 2 it's no more possible to go back to level 1 or 0
10 *
11 * @retval None
12 */
13 void FLASH_OB_RDPConfig(uint8_t OB_RDP)
14 {
15     FLASH_Status status = FLASH_COMPLETE;
16
17     /* Check the parameters */
18     assert_param(IS_OB_RDP(OB_RDP));
19
20     status = FLASH_WaitForLastOperation();
21
22     if(status == FLASH_COMPLETE)
23     {
24         *(__IO uint8_t*)OPTCR_BYTE1_ADDRESS = OB_RDP;
25     }
26 }
27 }
```

该函数根据输入参数设置 RDP 寄存器位为相应的级别，其注释警告了若配置成 OB_RDP_Level_2 将无法恢复。类似地，配置其它选项时也有相应的库函数，如 FLASH_OB_PCROP1Config、FLASH_OB_WRP1Config 分别用于设置要进行 PCROP 保护或 WRP 保护(写保护)的扇区。

3. 写入选项字节

调用上一步骤中的函数配置寄存器后，还要调用代码清单 51-3 中的 FLASH_OB_Launch 函数把寄存器的内容写入到选项字节中。

代码清单 51-3 写入选项字节

```
1 /**
2  * @brief Launch the option byte loading.
3  * @param None
4  * @retval FLASH_Status: The returned value can be: FLASH_BUSY, FLASH_ERROR_PROGRAM,
5  *           FLASH_ERROR_WRP, FLASH_ERROR_OPERATION or FLASH_COMPLETE.
6  */
7 FLASH_Status FLASH_OB_Launch(void)
8 {
9     FLASH_Status status = FLASH_COMPLETE;
10
11     /* Set the OPTSTRT bit in OPTCR register */
12     *(__IO uint8_t *)OPTCR_BYTE0_ADDRESS |= FLASH_OPTCR_OPTSTRT;
13
14     /* Wait for last operation to be completed */
```



```
15  status = FLASH_WaitForLastOperation();  
16  
17  return status;  
18 }
```

该函数设置 FLASH_OPTCR_OPTSTRT 位后调用了 FLASH_WaitForLastOperation 函数等待写入完成，并返回写入状态，若操作正常，它会返回 FLASH_COMPLETE。

51.4 实验：设置读写保护及解除

在本实验中我们将以实例讲解如何修改选项字节的配置，更改读保护级别、设置 PCROP 或写保护，最后把选项字节恢复默认值。

本实验要进行的操作比较特殊，在开发和调试的过程中都是在 SRAM 上进行的（使用 SRAM 启动方式）。例如，直接使用 FLASH 版本的程序进行调试时，如果该程序在运行后对扇区进行了写保护而没有解除的操作或者该解除操作不正常，此时将无法再给芯片的内部 FLASH 下载新程序，最终还是要使用 SRAM 自举的方式进行解除操作。所以在本实验中为便于修改选项字节的参数，我们统一使用 SRAM 版本的程序进行开发和学习，当 SRAM 版本调试正常后再改为 FLASH 版本。

关于在 SRAM 中调试代码的相关配置，请参考前面的章节。

注意：

若您在学习的过程中想亲自修改代码进行测试，请注意备份原工程代码。当芯片的 FLASH 被保护导致无法下载程序到 FLASH 时，可以下载本工程到芯片，并使用 SRAM 启动运行，即可恢复芯片至默认配置。但如果修改了读保护为级别 2，采用任何方法都无法恢复！（除了这个配置，其它选项都可以大胆地修改测试。）

51.4.1 硬件设计

本实验在 SRAM 中调试代码，因此把 BOOT0 和 BOOT1 引脚都使用跳线帽连接到 3.3V，使芯片从 SRAM 中启动。

51.4.2 软件设计

本实验的工程名称为“设置读写保护与解除”，学习时请打开该工程配合阅读，它是从“RAM 调试—多彩流水灯”工程改写而来的。为了方便展示及移植，我们把操作内部 FLASH 相关的代码都编写到“internalFlash_reset.c”及“internalFlash_reset.h”文件中，这些文件是我们自己编写的，不属于标准库的内容，可根据您的喜好命名文件。

1. 主要实验

- (1) 学习配置扇区写保护；
- (2) 学习配置读保护级别；
- (3) 学习如何恢复选项字节到默认配置；

2. 代码分析

配置扇区写保护

我们先以代码清单 51-4 中的设置与解除写保护过程来学习如何配置选项字节。

代码清单 51-4 配置扇区写保护

```
1
2 #define FLASH_WRP_SECTORS    (OB_WRP_Sector_0|OB_WRP_Sector_1)
3 __IO uint32_t SectorsWRPStatus = 0xFF;
4
5 /**
6  * @brief WriteProtect_Test,普通的写保护配置
7  * @param 运行本函数后会给扇区 FLASH_WRP_SECTORS 进行写保护,再重复一次会进行解写保护
8  * @retval None
9  */
10 void WriteProtect_Test(void)
11 {
12     FLASH_Status status = FLASH_COMPLETE;
13     {
14         /* 获取扇区的写保护状态 */
15         SectorsWRPStatus = FLASH_OB_GetWRP() & FLASH_WRP_SECTORS;
16
17         if (SectorsWRPStatus == 0x00)
18         {
19             /* 扇区已被写保护,执行解保护过程*/
20
21             /* 使能访问 OPTCR 寄存器 */
22             FLASH_OB_Unlock();
23
24             /* 设置对应的 nWRP 位,解除写保护 */
25             FLASH_OB_WRPConfig(FLASH_WRP_SECTORS, DISABLE);
26             status=FLASH_OB_Launch();
27             /* 开始对选项字节进行编程 */
28             if (status != FLASH_COMPLETE)
29             {
30                 FLASH_ERROR("对选项字节编程出错,解除写保护失败, status = %x",status);
31                 /* User can add here some code to deal with this error */
32                 while (1)
33                 {
34                 }
35             }
36             /* 禁止访问 OPTCR 寄存器 */
37             FLASH_OB_Lock();
38
39             /* 获取扇区的写保护状态 */
40             SectorsWRPStatus = FLASH_OB_GetWRP() & FLASH_WRP_SECTORS;
41
42             /* 检查是否配置成功 */
43             if (SectorsWRPStatus == FLASH_WRP_SECTORS)
44             {
45                 FLASH_INFO("解除写保护成功!");
46             }
47             else
48             {
49                 FLASH_ERROR("未解除写保护!");
50             }
51         }
52         else
53         { /* 若扇区未被写保护,开启写保护配置 */
54
55             /* 使能访问 OPTCR 寄存器 */
```



```
56     FLASH_OB_Unlock();
57
58     /*使能 FLASH_WRP_SECTORS 扇区写保护 */
59     FLASH_OB_WRPConfig(FLASH_WRP_SECTORS, ENABLE);
60
61     status=FLASH_OB_Launch();
62     /* 开始对选项字节进行编程 */
63     if (status != FLASH_COMPLETE)
64     {
65         FLASH_ERROR("对选项字节编程出错, 设置写保护失败, status = %x",status);
66         while (1)
67         {
68         }
69     }
70     /* 禁止访问 OPTCR 寄存器 */
71     FLASH_OB_Lock();
72
73     /* 获取扇区的写保护状态 */
74     SectorsWRPStatus = FLASH_OB_GetWRP() & FLASH_WRP_SECTORS;
75
76     /* 检查是否配置成功 */
77     if (SectorsWRPStatus == 0x00)
78     {
79         FLASH_INFO("设置写保护成功!");
80     }
81     else
82     {
83         FLASH_ERROR("设置写保护失败!");
84     }
85 }
86 }
87 }
```

本函数分成了两个部分，它根据目标扇区的状态进行操作，若原来扇区为非保护状态时就进行写保护，若为保护状态就解除保护。其主要操作过程如下：

- ❑ 调用 FLASH_OB_GetWRP 函数获取目标扇区的保护状态若扇区被写保护，则开始解除保护过程，否则开始设置写保护过程；
- ❑ 调用 FLASH_OB_Unlock 解锁选项字节的编程；
- ❑ 调用 FLASH_OB_WRPConfig 函数配置目标扇区关闭或打开写保护；
- ❑ 调用 FLASH_OB_Launch 函数把寄存器的配置写入到选项字节；
- ❑ 调用 FLASH_OB_GetWRP 函数检查是否配置成功；
- ❑ 调用 FLASH_OB_Lock 禁止修改选项字节。

恢复选项字节为默认值

当芯片被设置为读写保护时，这时给芯片的内部 FLASH 下载程序时，可能会出现图 51-5 和图 51-6 的擦除 FLASH 失败的错误提示。

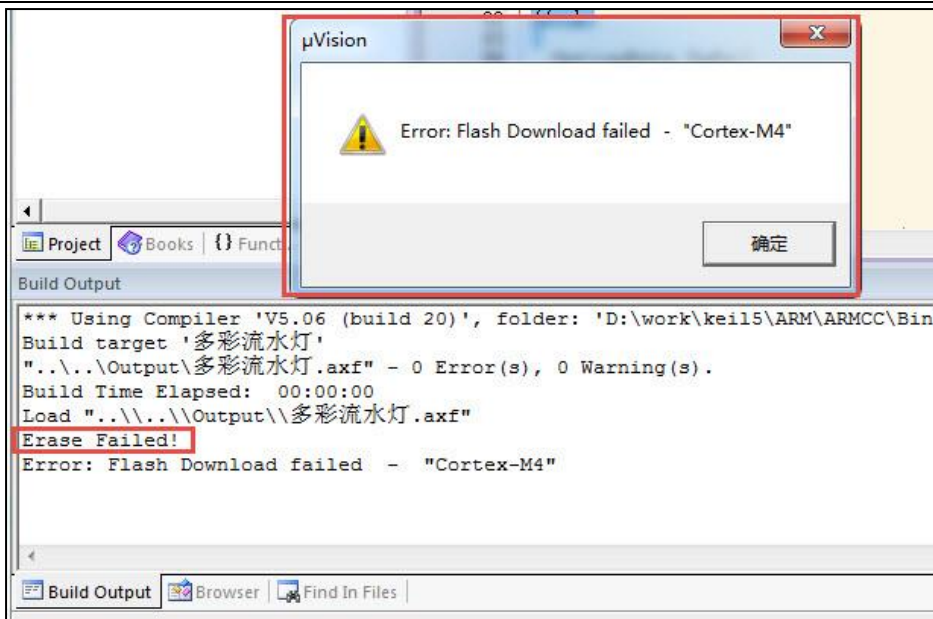


图 51-5 擦除失败提示

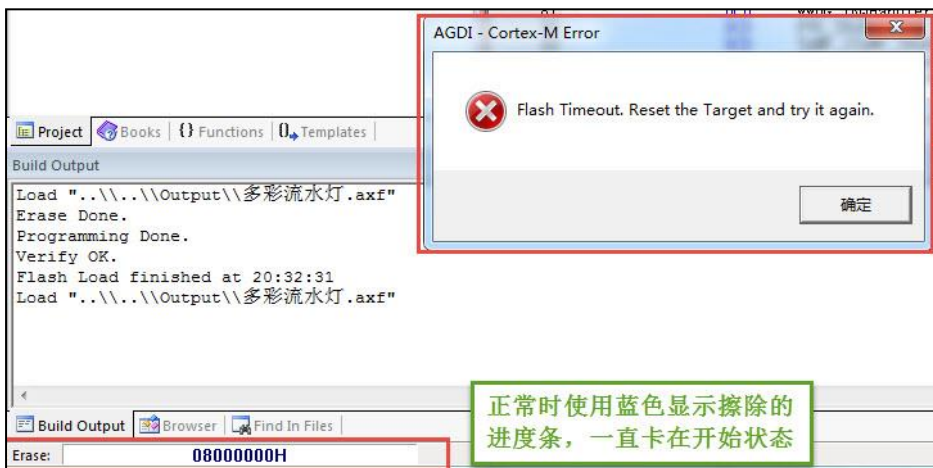


图 51-6 擦除进度条卡在开始状态

只要不是把读保护配置成了级别 2 保护，都可以使用 SRAM 启动运行代码清单 51-5 中的函数恢复选项字节为默认状态，使得 FLASH 下载能正常进行。

代码清单 51-5 恢复选项字节为默认值

```
1 // @brief  OPTCR register byte 0 (Bits[7:0]) base address
2 #define OPTCR_BYTE0_ADDRESS ((uint32_t)0x40023C14)
3
4 // @brief  OPTCR register byte 1 (Bits[15:8]) base address
5 #define OPTCR_BYTE1_ADDRESS ((uint32_t)0x40023C15)
6
7 // @brief  OPTCR register byte 2 (Bits[23:16]) base address
8 #define OPTCR_BYTE2_ADDRESS ((uint32_t)0x40023C16)
9
10 // @brief  OPTCR register byte 3 (Bits[31:24]) base address
11 #define OPTCR_BYTE3_ADDRESS ((uint32_t)0x40023C17)
12
13 // @brief  OPTCR1 register byte 0 (Bits[7:0]) base address
14 #define OPTCR1_BYTE2_ADDRESS ((uint32_t)0x40023C1A)
15
16 /**
17  * @brief  InternalFlash_Reset,恢复内部 FLASH 的默认配置
```

```
18  * @param None
19  * @retval None
20  */
21 int InternalFlash_Reset(void)
22 {
23     FLASH_Status status = FLASH_COMPLETE;
24
25     /* 使能访问选项字节寄存器 */
26     FLASH_OB_Unlock();
27
28     /* 擦除用户区域 (用户区域指程序本身没有使用的空间, 可以自定义) */
29     /* 清除各种 FLASH 的标志位 */
30     FLASH_ClearFlag(FLASH_FLAG_EOP | FLASH_FLAG_OPERR | FLASH_FLAG_WRPERR |
31                     FLASH_FLAG_PGAERR | FLASH_FLAG_PGPERR | FLASH_FLAG_PGSERR);
32
33     FLASH_INFO("\r\n");
34     FLASH_INFO("正在准备恢复的条件, 请耐心等待...");
35
36     //确保把读保护级别设置为 LEVEL1, 以便恢复
37     //必须是读保护级别由 LEVEL1 转为 LEVEL0 时才有效,
38     //否则修改无效
39     FLASH_OB_RDPConfig(OB_RDP_Level_1);
40
41     status=FLASH_OB_Launch();
42
43     status = FLASH_WaitForLastOperation();
44
45     //设置为 LEVEL0
46
47     FLASH_INFO("\r\n");
48     FLASH_INFO("正在擦除内部 FLASH 的内容, 请耐心等待...");
49
50     FLASH_OB_RDPConfig(OB_RDP_Level_0);
51
52     status =FLASH_OB_Launch();
53
54     //设置其它位为默认值
55     (*( __IO uint32_t *) (OPTCR_BYTE0_ADDRESS))=0xFFFFAAE9;
56     (*( __IO uint16_t *) (OPTCR1_BYTE2_ADDRESS))=0x0FFF;
57     status =FLASH_OB_Launch();
58
59     if (status != FLASH_COMPLETE) {
60         FLASH_ERROR("恢复选项字节默认值失败, 错误代码: status=%X", status);
61     } else {
62         FLASH_INFO("恢复选项字节默认值成功!");
63     }
64
65     //禁止访问
66     FLASH_OB_Lock();
67
68     return status;
69 }
```

这个函数进行了如下操作:

- ☐ 调用 FLASH_OB_Unlock 解锁选项字节的编程;
- ☐ 调用 FLASH_ClearFlag 函数清除所有 FLASH 异常状态标志;
- ☐ 调用 FLASH_OB_RDPConfig 函数设置为读保护级别 1, 以便后面能正常关闭 PCROP 模式;
- ☐ 调用 FLASH_OB_Launch 写入选项字节并等待读保护级别设置完毕;
- ☐ 调用 FLASH_OB_RDPConfig 函数把读保护级别降为 0;

- ❑ 调用 FLASH_OB_Launch 定稿选项字节并等待降级完毕，由于这个过程需要擦除内部 FLASH 的内容，等待的时间会比较长；
- ❑ 直接操作寄存器，使用 “(*(__IO uint32_t*)(OPTCR_BYTE0_ADDRESS))=0xFFFAAE9;” 和 “(*(__IO uint16_t*)(OPTCR1_BYTE2_ADDRESS))=0xFFFF;” 语句把 OPTCR 及 OPTCR1 寄存器与选项字节相关的位都恢复默认值；
- ❑ 调用 FLASH_OB_Launch 函数等待上述配置被写入到选项字节；
- ❑ 恢复选项字节为默认值操作完毕。

main 函数

最后来看看本实验的 main 函数，见。代码清单 51-6。

代码清单 51-6 main 函数

```
1 /**
2  * @brief 主函数
3  * @param 无
4  * @retval 无
5  */
6 int main(void)
7 {
8     /* LED 端口初始化 */
9     LED_GPIO_Config();
10    Debug_USART_Config();
11    LED_BLUE;
12
13    FLASH_INFO("本程序将会被下载到 STM32 的内部 SRAM 运行。");
14    FLASH_INFO("下载程序前，请确认把实验板的 BOOT0 和 BOOT1 引脚都接到 3.3V 电源处！！");
15
16    FLASH_INFO("\r\n");
17    FLASH_INFO("----这是一个 STM32 芯片内部 FLASH 解锁程序----");
18    FLASH_INFO("程序会把芯片的内部 FLASH 选项字节恢复为默认值");
19
20    #if 0 //工程调试、演示时使用，正常解除时不需要运行此函数
21        WriteProtect_Test(); //修改写保护位，仿真芯片扇区被设置成写保护的的环境
22    #endif
23
24    OptionByte_Info();
25
26    /*恢复选项字节到默认值，解除保护*/
27    if (InternalFlash_Reset()==FLASH_COMPLETE) {
28        FLASH_INFO("选项字节恢复成功，请把 BOOT0 和 BOOT1 引脚都连接到 GND，");
29        FLASH_INFO("然后随便找一个普通的程序，下载程序到芯片的内部 FLASH 进行测试");
30        LED_GREEN;
31    } else {
32        FLASH_INFO("选项字节恢复成功失败，请复位重试");
33        LED_RED;
34    }
35
36    OptionByte_Info();
37
38    while (1) {
39    }
40 }
41 }
```

在 main 函数中，主要是调用了 InternalFlash_Reset 函数把选项字节恢复成默认值，程序默认时没有调用 WriteProtect_Test 函数设置写保护，若您想观察实验现象，可修改条件编译的宏，使它加入到编译中。

3. 下载测试

把开发板的 BOOT0 和 BOOT1 引脚都使用跳线帽连接到 3.3V 电源处，使它以 SRAM 方式启动，然后用 USB 线连接开发板“USB TO UART”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板并复位运行，在串口调试助手可看到调试信息。程序运行后，请耐心等待至开发板亮绿灯或串口调试信息提示恢复完毕再给开发板断电，否则由于恢复过程被中断，芯片内部 FLASH 会处于保护状态。

芯片内部 FLASH 处于保护状态时，可重新下载本程序到开发板以 SRAM 运行恢复默认配置。